```
"""核心引擎：分段 → 日型分类 → 目标位阶梯 → 入场区 → 不做条件。"""
from __future__ import annotations

from datetime import datetime

from . import data as D

DAY_TYPES = {
    "classic_reversal": "经典反转日：伦敦扫一边后收回，纽约上午大概率反向修复",
    "rotation": "轮转日：伦敦困在区间内或两边都碰，方向不明",
    "trend": "单边日：三段同向，只等一次回调，不给回调就不做",
    "compression": "压缩日：全程横盘、参与度低，直接不做",
}


# ---------- 1. 分段 ----------
def build_sessions(bars, today: datetime, cfg: dict) -> dict:
    s = cfg["sessions"]
    a0, a1 = D.session_window(today, s["asia_start"], s["asia_end"], start_prev_d
    l0, l1 = D.session_window(today, s["london_start"], s["london_end"])
    return {
        "asia": D.session_stats(D.slice_session(bars, a0, a1)),
        "london": D.session_stats(D.slice_session(bars, l0, l1)),
        "asia_window": (a0.isoformat(), a1.isoformat()),
        "london_window": (l0.isoformat(), l1.isoformat()),
    }


# ---------- 2. 日型分类 ----------
def classify(sess: dict, atr20: float, cfg: dict) -> dict:
    asia, ldn = sess.get("asia"), sess.get("london")
    c = cfg["classification"]
    if not asia or not ldn:
        return {"type": "unknown", "bias": None, "why": "数据缺失"}

    swept_high = ldn["high"] > asia["high"]
    swept_low = ldn["low"] < asia["low"]
    depth = max(asia["range"], 1e-9)

    # 压缩日
    if ldn["range"] < c["compression_atr_ratio"] * atr20 and not (swept_high or s
        return {"type": "compression", "bias": None,
                "why": f"伦敦区间 {ldn['range']:.1f} < {c['compression_atr_ratio']
```

```
    if swept_high and swept_low:
        return {"type": "rotation", "bias": None, "why": "伦敦两边都扫，无干净的流动性

    if not swept_high and not swept_low:
        return {"type": "rotation", "bias": None, "why": "伦敦全程困在亚洲区间内"}

    # 单边突破：收盘站稳区间外
    if swept_high and ldn["close"] > asia["high"] + c["trend_break_ratio"] * dept
        return {"type": "trend", "bias": "long",
                "why": f"伦敦收在亚洲高点上方 {ldn['close'] - asia['high']:.1f} 点，站
    if swept_low and ldn["close"] < asia["low"] - c["trend_break_ratio"] * depth:
        return {"type": "trend", "bias": "short",
                "why": f"伦敦收在亚洲低点下方 {asia['low'] - ldn['close']:.1f} 点，站

    # 扫完收回 = 反转燃料
    reclaim = c["reclaim_threshold"] * depth
    if swept_high and ldn["close"] < asia["high"] - reclaim:
        return {"type": "classic_reversal", "bias": "short",
                "why": f"伦敦扫掉亚洲高点 {ldn['high'] - asia['high']:.1f} 点后收回区
    if swept_low and ldn["close"] > asia["low"] + reclaim:
        return {"type": "classic_reversal", "bias": "long",
                "why": f"伦敦扫掉亚洲低点 {asia['low'] - ldn['low']:.1f} 点后收回区间

    return {"type": "rotation", "bias": None, "why": "扫了边但没有干净收回，真假未定"}


# ---------- 3. 目标位阶梯 ----------
def target_ladder(sess: dict, bars_1h, ref_price: float, gx: dict, bias: str | No
    """固定优先级：伦敦 H/L → 亚洲 H/L → 亚洲中点 → 1H/4H 摆动位。
    同优先级内按距现价远近排序，最近的先成为目标。"""
    asia, ldn = sess.get("asia"), sess.get("london")
    cands: list[dict] = []
    if not asia:
        return []

    def add(price, label, prio):
        if price is None:
            return
        cands.append({"price": round(float(price), 2), "label": label, "priority"
                      "distance": round(abs(float(price) - ref_price), 2),
                      "side": "above" if price > ref_price else "below"})

    if ldn:
        add(ldn["high"], "伦敦盘高点", 1)
        add(ldn["low"], "伦敦盘低点", 1)
    add(asia["high"], "亚洲盘高点", 2)
    add(asia["low"], "亚洲盘低点", 2)
```

```
    add(asia["mid"], "亚洲区间中点", 3)

    h1, l1 = D.swing_points(bars_1h.tail(120))
    for p in h1[:3]:
        add(p, "1H 摆动高", 4)
    for p in l1[:3]:
        add(p, "1H 摆动低", 4)
    bars_4h = bars_1h.resample("4h").agg({"open": "first", "high": "max",
                                          "low": "min", "close": "last", "volume"
    h4, l4 = D.swing_points(bars_4h.tail(60))
    for p in h4[:2]:
        add(p, "4H 摆动高", 5)
    for p in l4[:2]:
        add(p, "4H 摆动低", 5)

    # GEX 墙作为附加标签（不改优先级，只提示阻力性质）
    for key, name in (("call_wall", "GEX Call Wall"), ("put_wall", "GEX Put Wall"
                      ("gamma_flip", "Gamma Flip")):
        if gx.get(key):
            add(gx[key], name, 6)

    if bias == "long":
        cands = [c for c in cands if c["side"] == "above"]
    elif bias == "short":
        cands = [c for c in cands if c["side"] == "below"]

    cands.sort(key=lambda x: (x["priority"], x["distance"]))
    # 去重：50 点内的同侧目标只保留优先级最高的
    out: list[dict] = []
    for c in cands:
        if not any(abs(c["price"] - o["price"]) < 15 for o in out):
            out.append(c)
    return out[:6]


# ---------- 4. 入场区（折价/溢价 fib） ----------
def entry_zone(sess: dict, bias: str | None, cfg: dict) -> dict | None:
    """用伦敦试探腿画区间：多头看折价区，空头看溢价区。
    深度超过 0.886 = 这笔交易不存在。"""
    if bias is None:
        return None
    ldn = sess.get("london")
    if not ldn:
        return None
    lo, hi = ldn["low"], ldn["high"]
    rng = hi - lo
    if rng <= 0:
```

```
        return None
    f_s, f_d = cfg["entry"]["fib_shallow"], cfg["entry"]["fib_deep"]
    if bias == "long":
        zone = (hi - f_d * rng, hi - f_s * rng)   # 折价区
        invalid = hi - f_d * rng
    else:
        zone = (lo + f_s * rng, lo + f_d * rng)   # 溢价区
        invalid = lo + f_d * rng
    return {
        "bias": bias,
        "leg_low": round(lo, 2), "leg_high": round(hi, 2),
        "zone_low": round(min(zone), 2), "zone_high": round(max(zone), 2),
        "invalidation": round(invalid, 2),
        "note": f"价格不进这个区就没有任何操作；穿过 {f_d} ({invalid:.2f}) 这笔作废。",
    }


def zone_reachable(zone: dict, price: float, atr20: float) -> tuple[bool, str]:
    """入场区离现价太远 = 大概率今天根本不触发，别守着它浪费一上午。"""
    if not zone:
        return False, "无入场区"
    if zone["zone_low"] <= price <= zone["zone_high"]:
        return True, "现价已在区内"
    dist = min(abs(price - zone["zone_low"]), abs(price - zone["zone_high"]))
    ok = dist <= 0.75 * atr20
    return ok, f"距入场区 {dist:.1f} 点 (ATR20 的 {dist/max(atr20,1e-9):.0%}) " + ("


# ---------- 5. 不做条件 ----------
def gates(day: dict, sess: dict, zone: dict | None, ladder: list[dict],
          gx: dict, premarket_price: float, cfg: dict, risk_state: dict,
          atr20: float = 0.0) -> list[dict]:
    f = cfg["filters"]
    g: list[dict] = []

    def add(name, ok, note):
        g.append({"gate": name, "pass": ok, "note": note})

    add("日型可交易", day["type"] in ("classic_reversal", "trend")
        or (day["type"] == "rotation" and not f["skip_on_rotation_day"]),
        DAY_TYPES.get(day["type"], "未知"))

    if day["type"] == "compression" and f["skip_on_compression_day"]:
        g[-1]["pass"] = False

    add("方向已定", day["bias"] is not None,
        f"偏向 {day['bias']}" if day["bias"] else "方向不明 → 少做或不做")
```

```
add("入场区已画好", zone is not None,
    f"{zone['zone_low']}-{zone['zone_high']}" if zone else "无有效试探腿")

reach_ok, reach_note = zone_reachable(zone, premarket_price, atr20) if zone e
add("入场区可达", reach_ok, reach_note)

# 开盘前目标已被打掉 = 当天的题已经被答完
hit = False
if ladder and f["skip_if_target_hit_premarket"]:
    nearest = ladder[0]
    hit = (nearest["side"] == "above" and premarket_price >= nearest["price"]
           (nearest["side"] == "below" and premarket_price <= nearest["price"]
add("最近目标位未被提前打掉", not hit,
    "开盘前已到目标 → 按高危时段处理，大概率不做" if hit else "目标位仍在前方")

# GEX 环境与打法是否冲突
if gx.get("regime") == "positive":
    add("GEX 环境", day["type"] != "trend",
        "正 gamma 压波动：假突破多。反转日加分，单边追突破减分")
elif gx.get("regime") == "negative":
    add("GEX 环境", True, "负 gamma 放大波动：允许顺势延伸，目标位可放远")
else:
    add("GEX 环境", True, "GEX 不可用，按中性处理")

# 盈亏比
if zone and ladder:
    entry = (zone["zone_low"] + zone["zone_high"]) / 2
    stop = zone["invalidation"]
    tgt = ladder[0]["price"]
    risk = abs(entry - stop)
    rr = abs(tgt - entry) / risk if risk > 0 else 0
    add("盈亏比达标", rr >= cfg["risk"]["min_rr"],
        f"到最近目标 {ladder[0]['label']} 为 {rr:.2f}R（门槛 {cfg['risk']['min_r

# 风控状态
add("风控状态允许", risk_state.get("allowed", True),
    risk_state.get("reason", "正常"))

add("参与度（盘中确认）", True,
    f"MNQ 5 分钟量 < {f['min_5m_volume_mnq']:,} 直接放弃该笔 —— 这一条要在盘中看，脚
add("订单流确认（盘中确认）", True,
    "footprint 需同时满足：折价区吸筹 + 主导权翻转 + 下一根 K 线价格推进。缺一个，这笔交

return g
```

```
def verdict(gates_list: list[dict]) -> tuple[str, list[str]]:
    fails = [g["gate"] for g in gates_list if not g["pass"]]
    if not fails:
        return "GO", []
    if len(fails) == 1 and "GEX 环境" in fails:
        return "REDUCED", fails
    return "NO-TRADE", fails
```